

Лекция 18. Retrofit

Большинство приложений для Android, представленных на рынке, подключаются к Интернету для выполнения сетевых операций, например для получения электронной почты, сообщений или другой информации с внутреннего сервера. Gmail, YouTube и Google Photos - это примеры приложений, которые подключаются к интернету для отображения пользовательских данных.

В этом уроке вы будете использовать библиотеки с открытым исходным кодом и библиотеки, созданные сообществом, для создания слоя данных и получения данных с внутреннего сервера. Это значительно упрощает получение данных, а также помогает приложению следовать лучшим практикам Android, например, выполнять операции в фоновом потоке. Кроме того, при медленном или недоступном интернете вы будете выводить сообщение об ошибке, что позволит информировать пользователя о любых проблемах с подключением к сети.

Необходимые условия

- Базовые знания о том, как создавать композитные функции.
- Базовые знания о том, как использовать компоненты `ViewModel` архитектуры Android.
- Базовые знания о том, как использовать `корутины` для выполнения длительных задач.
- Базовые знания о том, как добавлять зависимости в `build.gradle.kts`.

Что вы узнаете

- Что такое веб-сервис REST.
- Как с помощью библиотеки `Retrofit` подключиться к веб-сервису REST в интернете и получить ответ.
- Как использовать библиотеку `Serialization (kotlinx.serialization)` для разбора JSON-ответа на объекты данных.

Что вы будете делать

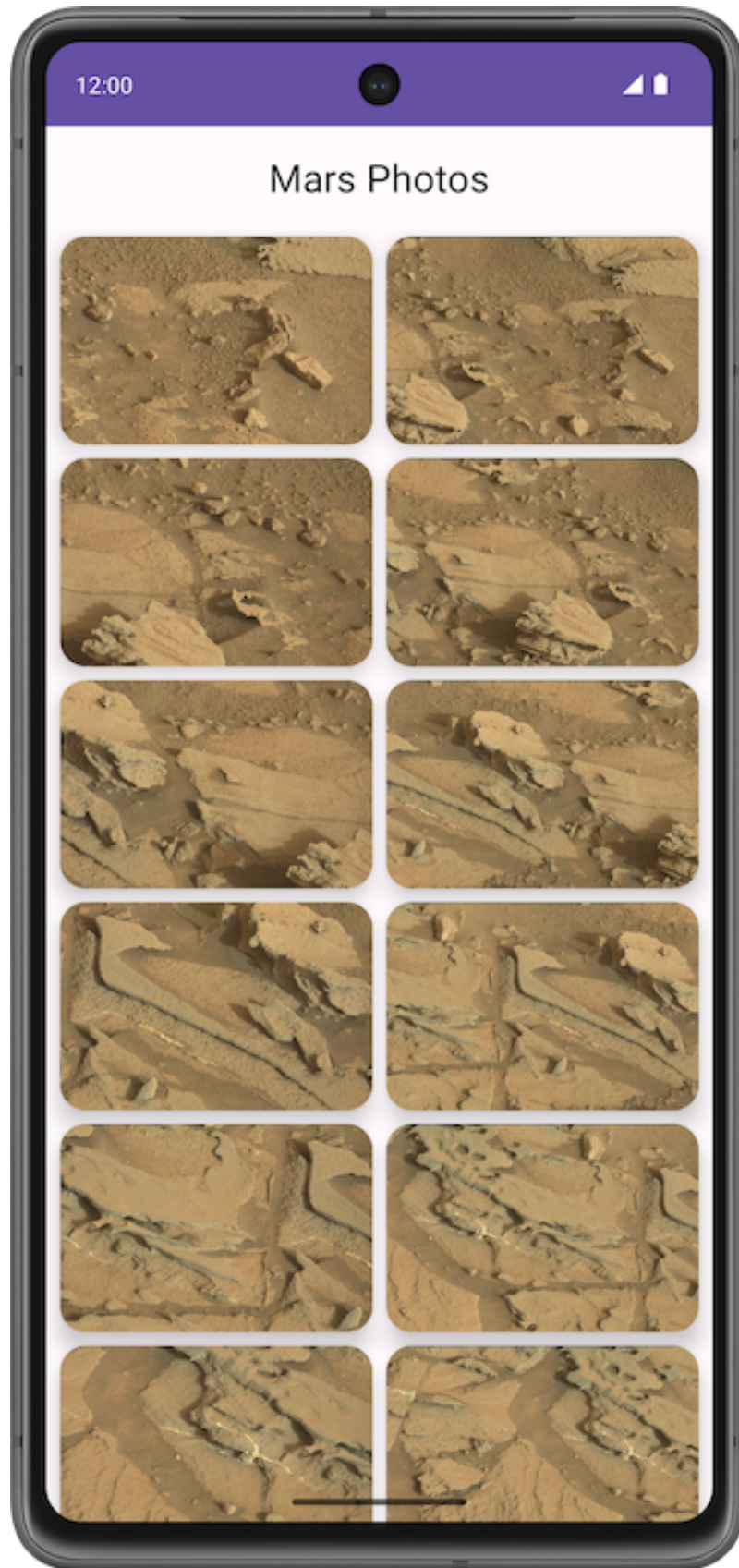
- Модифицировать стартовое приложение для выполнения API-запроса веб-сервиса и обработки ответа.
- Реализуйте слой данных для вашего приложения с помощью библиотеки `Retrofit`.
- Разберите JSON-ответ от веб-сервиса на список объектов данных вашего приложения с помощью библиотеки `kotlinx.serialization` и прикрепите его к состоянию пользовательского интерфейса.
- Используйте поддержку корутинов в `Retrofit` для упрощения кода.

Что вам понадобится

- Компьютер с Android Studio
- Стартовый код для приложения `Mars Photos`

Обзор приложения

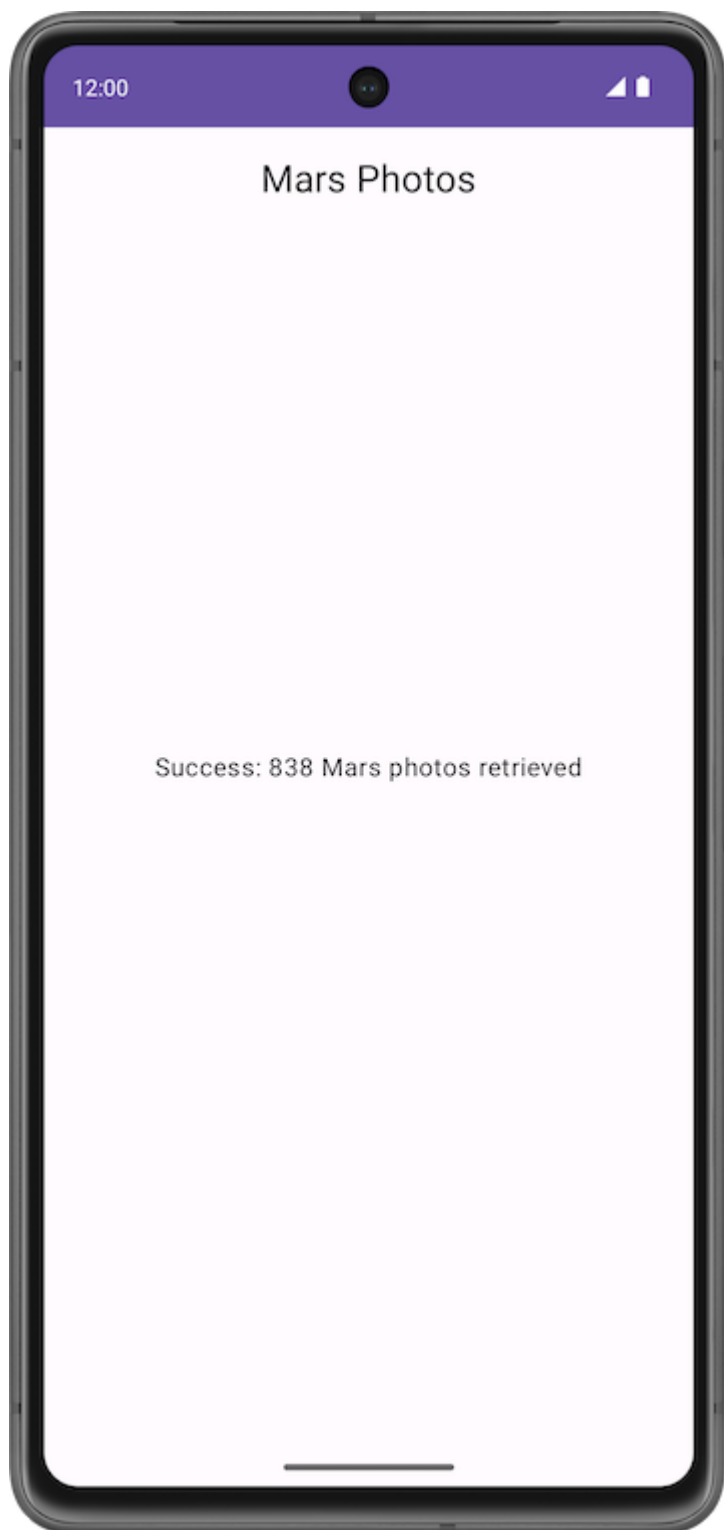
Вы работаете с приложением **Mars Photos**, которое показывает изображения поверхности Марса. Это приложение подключается к веб-службе для получения и отображения фотографий Марса. Изображения представляют собой реальные фотографии с Марса, сделанные марсоходами НАСА. Ниже приведен скриншот готового приложения, содержащего сетку изображений.



Примечание: предыдущее изображение - это снимок экрана финального приложения, которое вы создадите в конце этого раздела после дополнительных обновлений в последующих уроках.

Скриншот показан в этом уроке, чтобы дать вам лучшее представление об общей функциональности приложения.

Версия приложения, которую вы создадите в этом учебном уроке, не будет иметь большого количества визуальных эффектов. В этом руководстве основное внимание уделено части приложения, связанной с уровнем данных, для подключения к Интернету и загрузки необработанных данных о недвижимости с помощью веб-сервиса. Чтобы убедиться, что приложение правильно получает и анализирует эти данные, вы можете вывести количество фотографий, полученных с внутреннего сервера, в текстовый компонент.



Изучите стартовое приложение Mars Photos

- Загрузите стартовый код
- Чтобы начать работу, загрузите стартовый код:

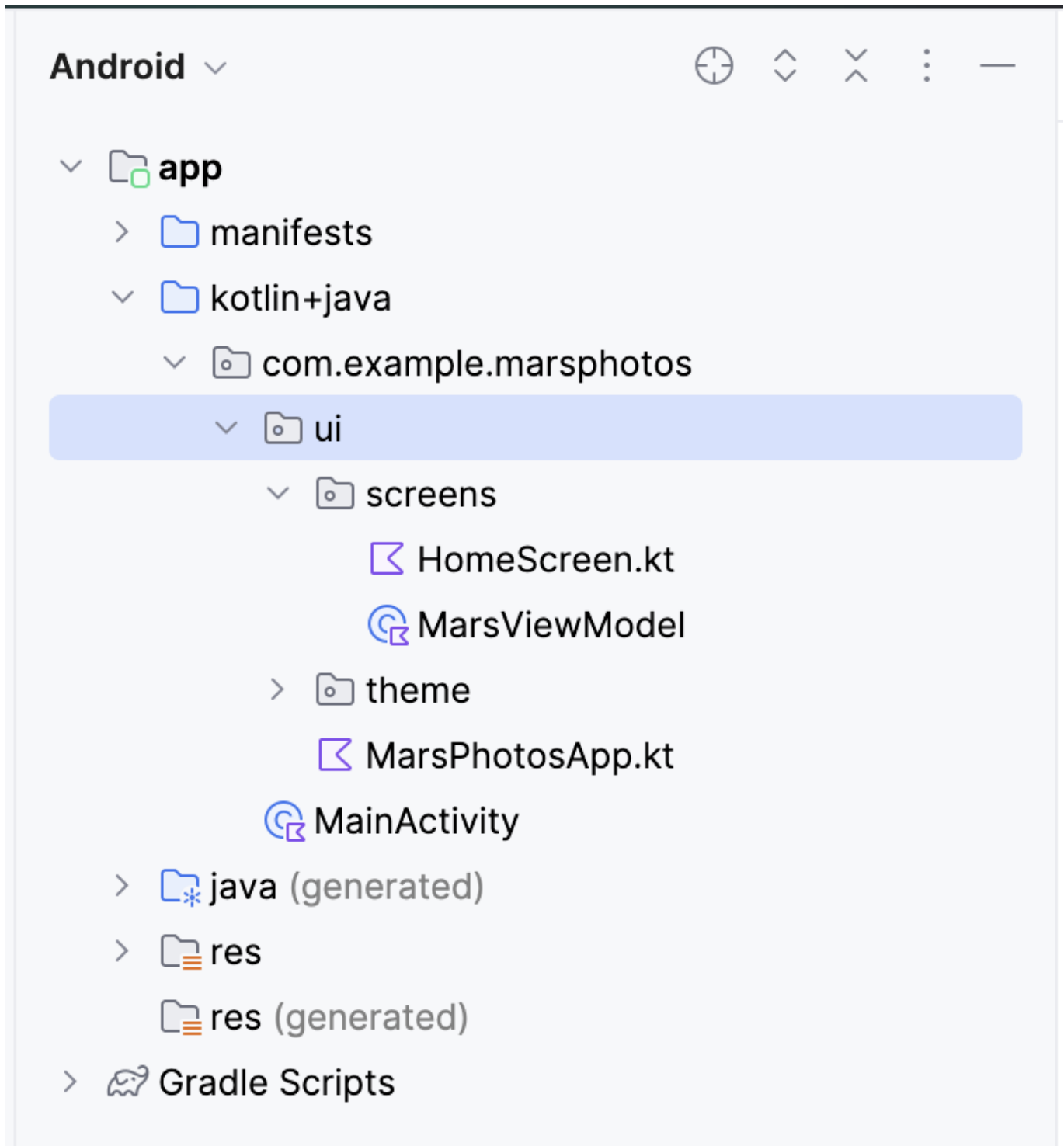
```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-mars-photos.git
$ cd basic-android-kotlin-compose-training-mars-photos
$ git checkout starter
```

Запуск стартового кода

- Откройте загруженный проект в Android Studio.

Имя папки проекта - `basic-android-kotlin-compose-training-mars-photos`.

В панели Android разверните приложение > kotlin + java. Обратите внимание, что в приложении есть папка пакетов под названием `ui`. Это слой пользовательского интерфейса приложения.



- Запустите приложение. Когда вы скомпилируете и запустите приложение, вы увидите следующий экран с текстом-заполнителем в центре. В конце этого урока вы обновите этот текст-заполнитель, указав количество полученных фотографий.



Прогулка по стартовому коду

В этом задании вы познакомитесь со структурой проекта. В следующих списках представлены важные файлы и папки проекта.

- `ui\MarsPhotosApp.kt`:

Этот файл содержит компонент `MarsPhotosApp`, который отображает содержимое на экране, например, верхнюю панель приложений и компонент `HomeScreen`. Текст-заполнитель из предыдущего шага отображается в этом композите. В следующем уроке этот компонент отображает данные, полученные с внутреннего сервера фотографий Марса.

- screens\MarsViewModel.kt:

Этот файл представляет собой соответствующую модель представления для **MarsPhotosApp**. Этот класс содержит свойство **MutableState** с именем **marsUiState**. Обновление значения этого свойства приводит к обновлению текста-заполнителя, отображаемого на экране.

Метод **getMarsPhotos()** обновляет ответное сообщение placeholder. Позже в уроке вы используете этот метод для отображения данных, полученных с сервера. Цель этого урока - обновить **MutableState** внутри **ViewModel**, используя данные, полученные из интернета.

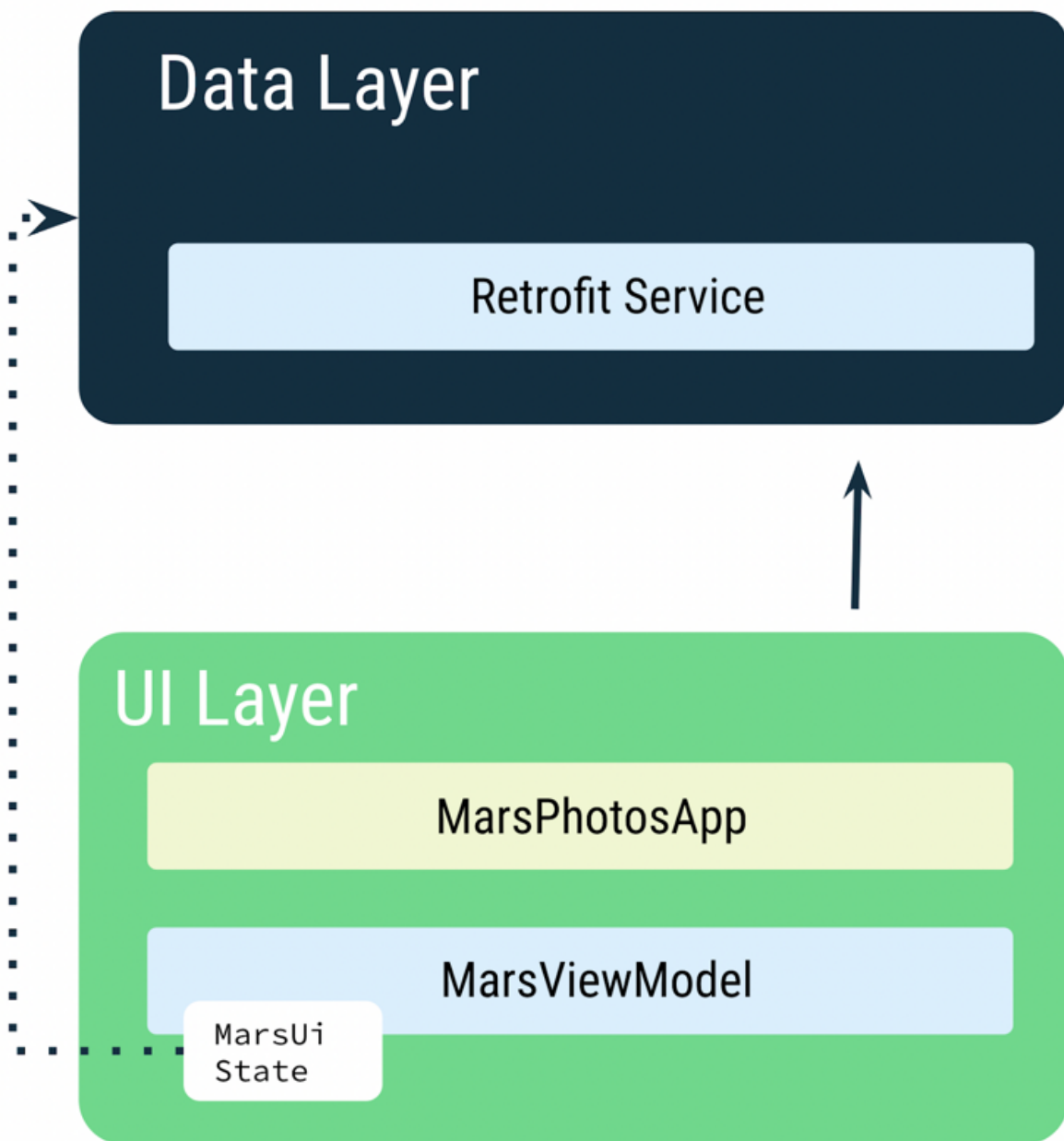
- screens\HomeScreen.kt:

Этот файл содержит композиты **HomeScreen** и **ResultScreen**. Экран результатов имеет простую компоновку **Box**, которая отображает значение **marsUiState** в виде текстового компонента.

- MainActivity.kt: Единственная задача этой активности - загрузить **ViewModel** и отобразить композит **MarsPhotosApp**.

Введение в веб-сервисы

В этом уроке вы создаете слой для сетевого сервиса, который взаимодействует с внутренним сервером и получает необходимые данные. Для реализации этой задачи вы используете стороннюю библиотеку **Retrofit**. Подробнее о ней вы узнаете позже. Модель **ViewModel** взаимодействует со слоем данных, а остальная часть приложения прозрачна для этой реализации.

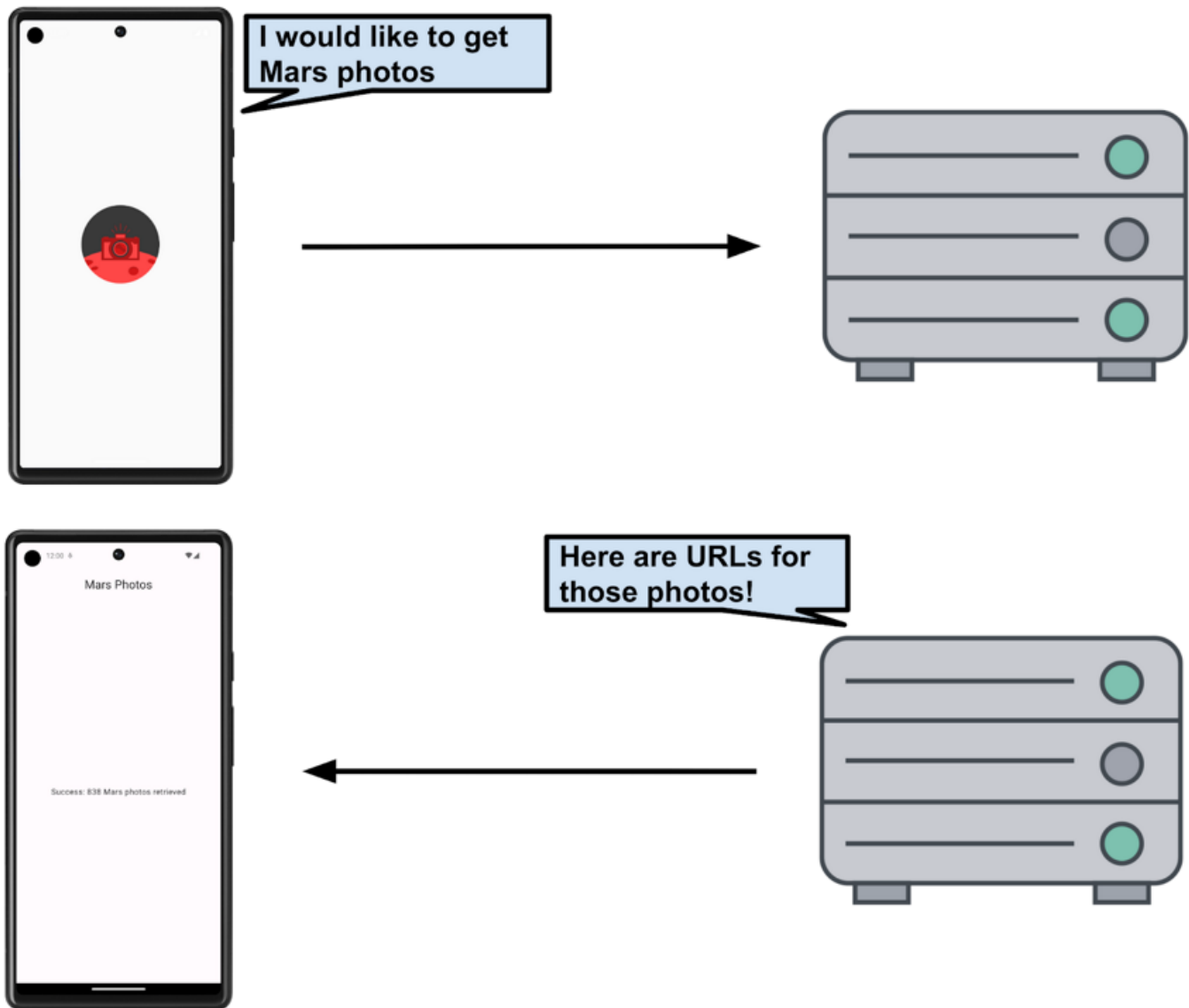


Модель **MarsViewModel** отвечает за выполнение сетевого вызова для получения данных о фотографиях Марса. В **ViewModel** вы используете **MutableState** для обновления пользовательского интерфейса приложения при изменении данных.

Примечание: В последующих уроках вы добавите репозиторий в слой данных. Репозиторий будет взаимодействовать с сервисом **Retrofit** для получения данных. Репозиторий отвечает за предоставление данных остальным частям приложения.

Веб-сервисы и Retrofit

Данные о фотографиях Марса хранятся на веб-сервере. Чтобы получить эти данные в вашем приложении, необходимо установить соединение и взаимодействовать с сервером через Интернет.



Примечание: В этом уроке вы получаете только количество записей (URL), а не фотографии Марса. В следующем уроке вы получите фотографии Марса и отобразите их в виде сетки.

Большинство современных веб-серверов запускают веб-сервисы, используя распространенную веб-архитектуру без статических данных, известную как **REST**, что расшифровывается как REpresentational State Transfer. Веб-сервисы, использующие эту архитектуру, называются **RESTful-сервисами**.

Запросы к веб-сервисам **RESTful** выполняются стандартным образом, через унифицированные идентификаторы ресурсов (URI). URI идентифицирует ресурс на сервере по имени, не указывая его местоположение или способ доступа к нему. Например, в приложении к этому уроку вы получаете URL-адреса изображений, используя следующий URI сервера. (На этом сервере хранятся как недвижимость Марса, так и фотографии Марса):

android-kotlin-fun-mars-server.appspot.com

Примечание: К вышеупомянутому серверу обращается другой пример приложения, где демонстрируется недвижимость Марса, поэтому у этого сервера есть две разные конечные точки: одна для недвижимости Марса и одна для фотографий. В этом курсе вы используете сервер для получения фотографий с Марса.

URL (Uniform Resource Locator) - это подмножество URI, которое указывает, где существует ресурс, и механизм его получения.

Например:

Следующий URL-адрес позволяет получить список доступных объектов недвижимости на Марсе:

Следующий URL-адрес позволяет получить список фотографий Марса:

Эти URL ссылаются на идентифицированный ресурс, например `/realestate` или `/photos`, который можно получить по протоколу Hypertext Transfer Protocol (`http:`) из сети. В этом уроке вы используете конечную точку `/photos`. Конечная точка - это URL-адрес, позволяющий получить доступ к веб-службе, запущенной на сервере.

Примечание: привычный веб-адрес на самом деле является типом URI. В этом уроке URL и URI используются как взаимозаменяемые.

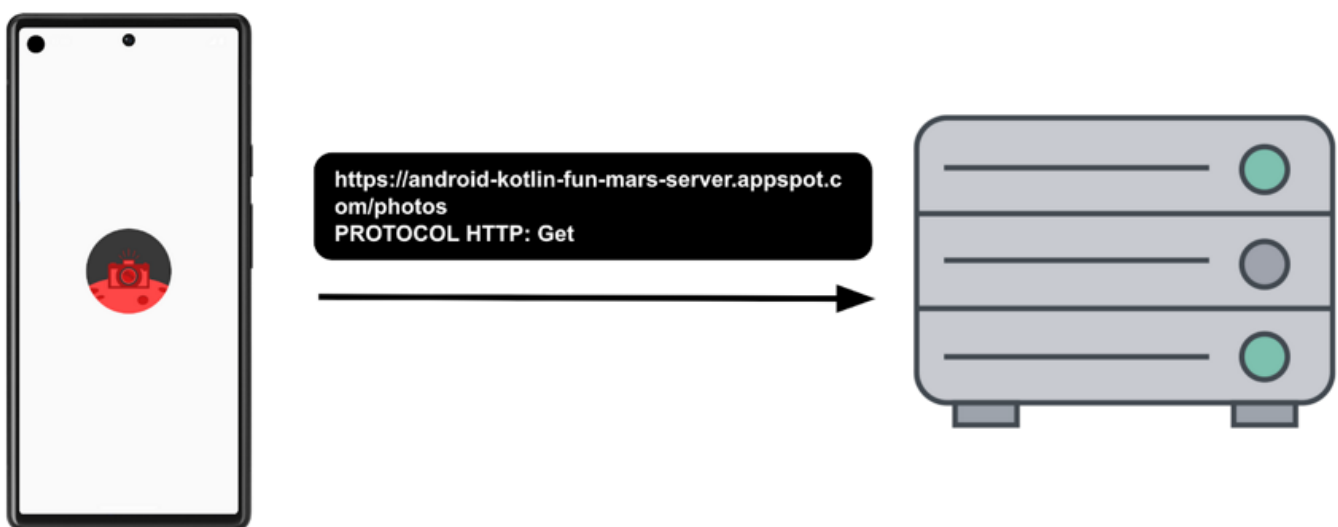
Запрос веб-сервиса

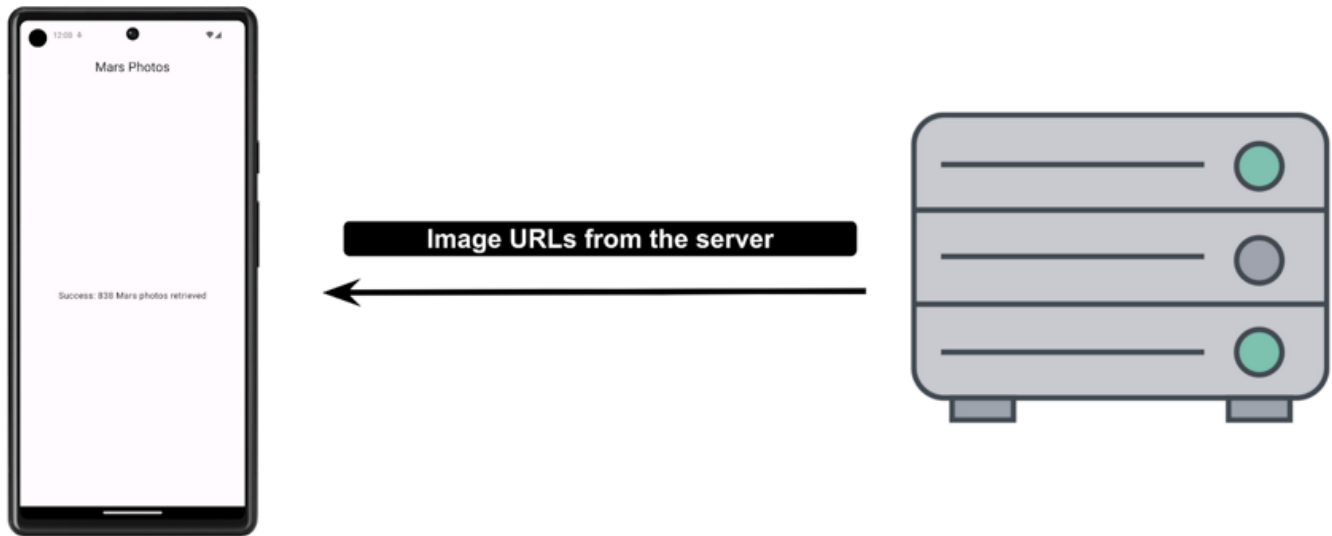
Каждый запрос веб-сервиса содержит URI и передается на сервер по тому же протоколу HTTP, который используется веб-браузерами, например Chrome. HTTP-запросы содержат операцию, указывающую серверу, что делать.

К распространенным операциям HTTP относятся:

- GET для получения данных с сервера.
- POST для создания новых данных на сервере.
- PUT - обновление существующих данных на сервере.
- DELETE - для удаления данных с сервера.

Ваше приложение делает HTTP GET-запрос к серверу для получения информации о фотографиях Марса, а затем сервер возвращает ответ вашему приложению, включая URL-адреса изображений.





Ответ от веб-сервиса оформляется в одном из распространенных форматов данных, например **XML** (eXtensible Markup Language) или **JSON** (JavaScript Object Notation). Формат JSON представляет структурированные данные в виде пар ключ-значение. Приложение взаимодействует с REST API с помощью JSON, о котором вы узнаете в одном из следующих заданий.

В этом задании вы устанавливаете сетевое соединение с сервером, взаимодействуете с ним и получаете ответ в формате JSON. Вы будете использовать внутренний сервер, который уже написан для вас. В этом уроке вы используете библиотеку **Retrofit**, библиотеку стороннего разработчика, для связи с внутренним сервером.

Внешние библиотеки

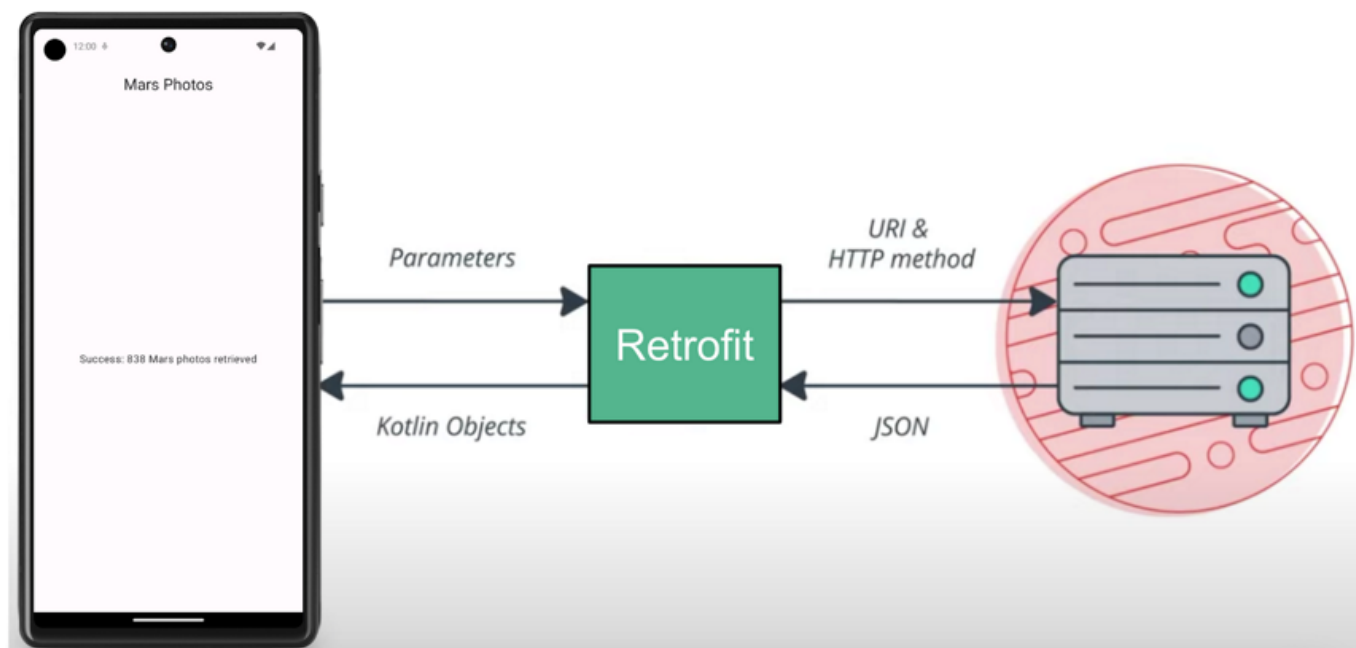
Внешние библиотеки, или библиотеки сторонних разработчиков, представляют собой расширения основных API-интерфейсов Android. Библиотеки, которые вы будете использовать в этом курсе, имеют открытый исходный код, разрабатываются сообществом и поддерживаются коллективным вкладом огромного сообщества Android по всему миру. Эти ресурсы помогают разработчикам Android, таким как вы, создавать лучшие приложения.

Предупреждение: Использование библиотек, разработанных и поддерживаемых сообществом, может значительно сэкономить время. Однако выбирать эти библиотеки нужно с умом, потому что ваше приложение в конечном итоге несет ответственность за то, что делает код в этих библиотеках.

Библиотека Retrofit

Библиотека **Retrofit**, которую вы используете в этом уроке для взаимодействия с RESTful веб-сервисом Mars, является хорошим примером хорошо поддерживаемой и обслуживаемой библиотеки. Об этом можно судить, заглянув на ее страницу на GitHub и просмотрев открытые и закрытые проблемы (некоторые из них являются запросами функций). Если разработчики регулярно решают проблемы и отвечают на запросы о возможностях, то библиотека, скорее всего, хорошо поддерживается и является хорошим кандидатом для использования в приложении. Вы также можете обратиться к документации Retrofit, чтобы узнать больше о библиотеке.

Библиотека Retrofit взаимодействует с REST-бэкендом. Она генерирует код, но вам нужно предоставить URI для веб-сервиса на основе параметров, которые мы ему передаем. Подробнее об этом в последующих разделах.



Добавление зависимостей Retrofit

Android Gradle позволяет добавлять внешние библиотеки в проект. Помимо зависимости от библиотеки, вам также нужно указать репозиторий, в котором она размещена.

- Откройте gradle-файл `build.gradle.kts` на уровне модуля (Module :app).
- В разделе зависимостей добавьте следующие строки для библиотек Retrofit:

```
// Ретрофит
implementation("com.squareup.retrofit2:retrofit:2.9.0")
// Ретрофит со скалярным конвертером
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

Эти две библиотеки работают вместе. Первая зависимость - это сама библиотека Retrofit2, а вторая - скалярный конвертер Retrofit. Retrofit2 - это обновленная версия библиотеки Retrofit. Этот скалярный конвертер позволяет Retrofit возвращать результат JSON в виде строки. JSON - это формат для хранения и передачи данных между клиентом и сервером. Вы узнаете о JSON в одном из следующих разделов.

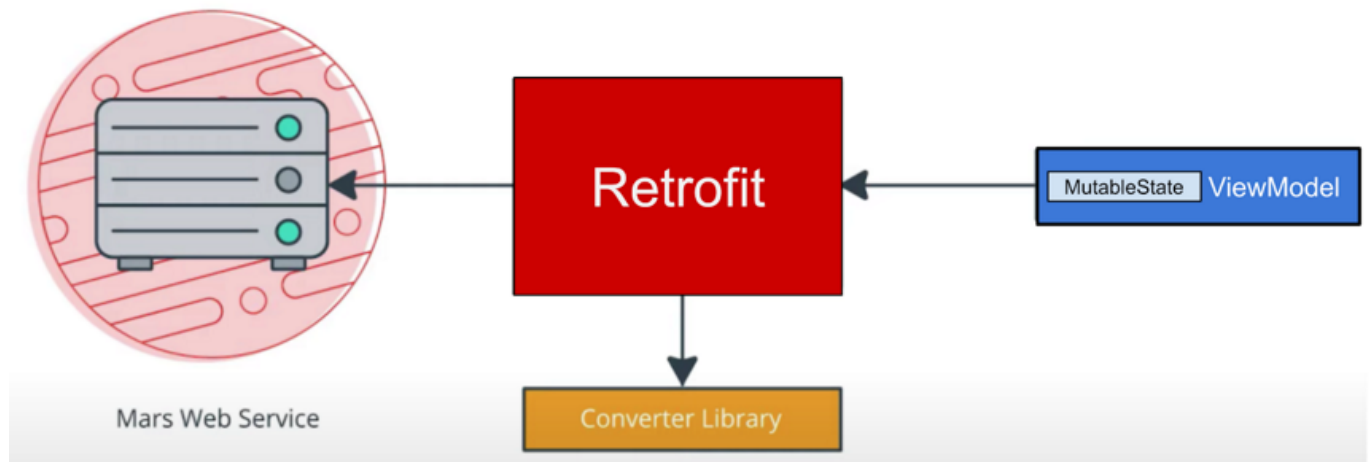
- Нажмите **Sync Now**, чтобы пересобрать проект с новыми зависимостями.

Подключение к Интернету

Вы используете библиотеку Retrofit для связи с веб-сервисом Mars и отображения необработанного JSON-ответа в виде строки. Заполнитель `Text` отображает либо возвращаемую строку JSON-ответа, либо сообщение об ошибке подключения.

Retrofit создает сетевой API для приложения на основе содержимого веб-сервиса. Он получает данные из веб-сервиса и направляет их через отдельную библиотеку-конвертер, которая умеет декодировать

данные и возвращать их в виде объектов, например String. В Retrofit встроена поддержка популярных форматов данных, таких как XML и JSON. В конечном итоге Retrofit создает код для вызова и использования этого сервиса за вас, включая такие важные детали, как выполнение запросов в фоновых потоках.



В этой задаче вы добавляете слой данных в проект Mars Photos, который ваша ViewModel использует для связи с веб-сервисом. Вы реализуете API сервиса Retrofit, выполнив следующие шаги:

- Создайте источник данных, класс `MarsApiService`.
- Создайте объект Retrofit с базовым URL и фабрикой конвертеров для преобразования строк.
- Создайте интерфейс, который объясняет, как Retrofit общается с веб-сервером.
- Создайте сервис Retrofit и предоставьте экземпляры api-сервиса остальным частям приложения.

Выполните описанные выше шаги:

- Щелкните правой кнопкой мыши на пакете `com.example.marsphotos` в панели проекта Android и выберите `New > Package`.
- Во всплывающем окне добавьте `network` в конец предложенного имени пакета.
- Создайте новый Kotlin-файл под новым пакетом. Назовите его `MarsApiService`.
- Откройте файл `network/MarsApiService.kt`.
- Добавьте следующую константу для базового URL веб-сервиса.

```
private const val BASE_URL = "https://android-kotlin-fun-mars-server.appspot.com"
```

- Добавьте конструктор Retrofit чуть ниже этой константы, чтобы построить и создать объект Retrofit.

```
import retrofit2.Retrofit

private val retrofit = Retrofit.Builder()
```

Чтобы создать API веб-сервиса, Retrofit нужен базовый URI веб-сервиса и фабрика конвертеров. Конвертер указывает Retrofit, что делать с данными, полученными от веб-сервиса. В данном случае вы

хотите, чтобы Retrofit получил от веб-сервиса ответ в формате JSON и вернул его в виде строки. В Retrofit есть конвертер `ScalarsConverter`, который поддерживает строки и другие примитивные типы.

- Вызовите `addConverterFactory()` в конструкторе с экземпляром `ScalarsConverterFactory`.

```
import retrofit2.converter.scalars.ScalarsConverterFactory

private val retrofit = Retrofit.Builder()
    .addConverterFactory(ScalarsConverterFactory.create())
```

- Добавьте базовый URL для веб-сервиса с помощью метода `baseUrl()`.
- Вызовите `build()`, чтобы создать объект Retrofit.

```
private val retrofit = Retrofit.Builder()
    .addConverterFactory(ScalarsConverterFactory.create())
    .baseUrl(BASE_URL)
    .build()
```

- Ниже вызова конструктора Retrofit определите интерфейс `MarsApiService`, который определяет, как Retrofit общается с веб-сервером с помощью HTTP-запросов.

```
interface MarsApiService {
}
```

- Добавьте в интерфейс `MarsApiService` функцию `getPhotos()` для получения строки ответа от веб-сервиса.

```
interface MarsApiService {
    fun getPhotos()
}
```

- Используйте аннотацию `@GET`, чтобы сообщить Retrofit, что это GET-запрос, и указать конечную точку для этого метода веб-сервиса. В данном случае конечной точкой являются фотографии. Как упоминалось в предыдущем задании, в этом уровне вы будете использовать конечную точку `/photos`.

```
import retrofit2.http.GET

interface MarsApiService {
    @GET("photos")
    fun getPhotos()
}
```

Когда вызывается метод `getPhotos()`, Retrofit добавляет фотографии конечной точки к базовому URL, который вы определили в конструкторе Retrofit, чтобы начать запрос.

- Добавьте тип возвращаемого значения функции в String.

```
interface MarsApiService {  
    @GET("photos")  
    fun getPhotos(): String  
}
```

Объявления объектов

В Kotlin объявления объектов используются для объявления **объектов-синглтонов**. Паттерн синглтон обеспечивает создание одного и только одного экземпляра объекта и одну глобальную точку доступа к нему. Инициализация объекта безопасна для потоков и выполняется при первом обращении.

Ниже приведен пример объявления объекта и доступа к нему. Объявление объекта всегда имеет имя, следующее за ключевым словом `object`.

Example:

```
// Example for Object declaration, do not copy over  
  
object SampleDataProvider {  
    fun register(provider: SampleProvider) {  
        // ...  
    }  
  
    // ...  
}  
  
// To refer to the object, use its name directly.  
SampleDataProvider.register(...)
```

Предупреждение: Паттерн синглтонов не является рекомендуемой практикой. Синглтоны представляют собой глобальные состояния, которые трудно предсказать, особенно в тестах. Объекты должны сами определять, какие зависимости им нужны, а не описывать, как их создавать. Используйте **инъекцию зависимостей** (DI) вместо паттерна синглтонов. О том, как реализовать инъекцию зависимостей, вы узнаете в одном из следующих уроков.

Вызов функции `create()` для объекта Retrofit требует больших затрат памяти, скорости и производительности. Приложению нужен только один экземпляр сервиса Retrofit API, поэтому вы открываете сервис для остальной части приложения с помощью объявления объектов.

- За пределами объявления интерфейса `MarsApiService` определите публичный объект `MarsApi` для инициализации сервиса Retrofit. Этот объект является публичным объектом-синглтоном, к которому могут обращаться остальные части приложения.

```
object MarsApi {}
```

- Внутри объявления объекта `MarsApi` добавьте лениво инициализируемое свойство объекта `retrofit` с именем `retrofitService` типа `MarsApiService`. Вы делаете эту ленивую инициализацию, чтобы убедиться, что он будет инициализирован при первом использовании. Не обращайте внимания на ошибку, которую вы исправите в следующих шагах.

Примечание: Помните, что «ленивая инициализация» - это когда создание объекта намеренно откладывается до тех пор, пока он действительно не понадобится, чтобы избежать лишних вычислений или использования других вычислительных ресурсов. В Kotlin есть первоклассная поддержка ленивой инстанцииции.

```
object MarsApi {  
    val retrofitService : MarsApiService by lazy {}  
}
```

- Инициализируйте переменную `retrofitService` с помощью метода `retrofit.create()` с интерфейсом `MarsApiService`.

```
object MarsApi {  
    val retrofitService : MarsApiService by lazy {  
        retrofit.create(MarsApiService::class.java)  
    }  
}
```

- Настройка Retrofit завершена! Каждый раз, когда ваше приложение вызывает `MarsApi.retrofitService`, вызывающий обращается к одному и тому же синглтону объекта Retrofit, реализующего `MarsApiService`, который создается при первом обращении. В следующей задаче вы используете реализованный вами объект Retrofit.

Вызов веб-сервиса в MarsViewModel

На этом шаге вы реализуете метод `getMarsPhotos()`, который вызывает REST-сервис и затем обрабатывает возвращаемую строку JSON.

Примечание: Рекомендуемый подход - вызывать веб-сервис из репозитория, что позволяет изолировать слой данных от остальной части приложения. В одном из последующих заданий вы добавите репозиторий в свое приложение.

ViewModelScope

`ViewModelScope` - это встроенная область выполнения корутин, определенная для каждой `ViewModel` в вашем приложении. Любая программа, запущенная в этой области, автоматически отменяется, если `ViewModel` очищается.

Вы можете использовать `viewModelScope` для запуска корутины и выполнения запроса веб-сервиса в фоновом режиме. Поскольку `viewModelScope` принадлежит `ViewModel`, запрос будет выполняться даже при изменении конфигурации приложения.

В файле `MarsApiService.kt` сделайте функцию `getPhotos()` приостановленной, чтобы сделать ее асинхронной и не блокировать вызывающий поток. Вы вызываете эту функцию изнутри `viewModelScope`.

```
@GET("photos")
suspend fun getPhotos(): String
```

- Откройте файл `ui/screens/MarsViewModel.kt`. Прокрутите вниз до метода `getMarsPhotos()`.
- Удалите строку, которая устанавливает статус ответа «Установите ответ API Mars здесь!», чтобы метод `getMarsPhotos()` был пустым.

```
private fun getMarsPhotos() {}
```

- Внутри `getMarsPhotos()` запустите корутину с помощью `viewModelScope.launch`.

```
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.launch

private fun getMarsPhotos() {
    viewModelScope.launch {}
}
```

- Внутри `viewModelScope` используйте объект-синглтон `MarsApi` для вызова метода `getPhotos()` из интерфейса `retrofitService`. Сохраните возвращаемый ответ в `val` под названием `listResult`.

```
import com.example.marsphotos.network.MarsApi

viewModelScope.launch {
    val listResult = MarsApi.retrofitService.getPhotos()
}
```

- Присвойте результат, только что полученный от внутреннего сервера, объекту `marsUiState`. `marsUiState` - это изменяемый объект состояния, который представляет статус последнего веб-запроса.

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = listResult
```

- Запустите приложение. Заметьте, что приложение сразу же закрывается и может отобразить или не отобразить всплывающее окно с ошибкой. Это сбой приложения. Перейдите на вкладку Logcat в Android Studio и обратите внимание на ошибку в журнале, которая начинается со строки, подобной этой: «----- начало сбоя».

```
----- beginning of crash
22803-22865/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION:
OkHttp Dispatcher
    Process: com.example.android.marsphotos, PID: 22803
    java.lang.SecurityException: Permission denied (missing INTERNET permission?)
...
```

Это сообщение об ошибке указывает на то, что приложению, возможно, не хватает разрешений INTERNET. В следующей задаче описано, как добавить разрешения на доступ в интернет для приложения и решить эту проблему.

Добавление разрешения на доступ в Интернет и обработка исключений

Разрешения в Android

Цель разрешений в Android - защитить конфиденциальность пользователя Android. Приложения для Android должны объявлять или запрашивать разрешения на доступ к конфиденциальным данным пользователя, таким как контакты, журналы вызовов и некоторые системные функции, например, камера или интернет.

Чтобы ваше приложение могло получить доступ к Интернету, ему необходимо разрешение INTERNET. Подключение к Интернету создает проблемы с безопасностью, поэтому приложения по умолчанию не подключаются к Интернету. Вам нужно явно объявить, что приложению нужен доступ к интернету. Это объявление считается обычным разрешением.

На этом шаге ваше приложение объявляет разрешения, которые ему требуются, путем включения тегов `<uses-permission>` в файл AndroidManifest.xml.

- Откройте файл `manifests/AndroidManifest.xml`. Добавьте эту строку непосредственно перед тегом `<application>`:

```
<uses-permission android:name="android.permission.INTERNET" />
```

- Скомпилируйте и запустите приложение снова.

Если у вас работает подключение к Интернету, вы увидите текст JSON, содержащий данные, связанные с фотографиями Марса. Обратите внимание, как `id` и `img_src` повторяются для каждой записи изображения.





```
3302-3302/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION: main
Process: com.example.android.marsphotos, PID: 3302
```

Исключения

Исключения - это ошибки, которые могут возникнуть во время выполнения, а не во время компиляции, и они резко завершают работу приложения без уведомления пользователя. Это может привести к ухудшению качества работы пользователя. Обработка исключений - это механизм, с помощью которого вы предотвращаете внезапное завершение работы приложения и решаете ситуацию удобным для пользователя способом.

Причиной исключений может быть простое деление на ноль или ошибка сетевого подключения. Эти исключения похожи на `IllegalArgumentException`, о котором шла речь в предыдущем уроке.

Примеры возможных проблем при подключении к серверу включают следующее:

- URL или URI, используемый в API, неверен.
- Сервер недоступен, и приложение не смогло к нему подключиться.
- Проблема с сетевой задержкой.
- Плохое или отсутствующее интернет-соединение на устройстве.

Эти исключения не могут быть обработаны во время компиляции, но вы можете использовать блок `try-catch` для обработки исключения во время выполнения.

Пример синтаксиса блока `try-catch`

```
try {  
    // some code that can cause an exception.  
}  
catch (e: SomeException) {  
    // handle the exception to avoid abrupt termination.  
}
```

Внутри блока `try` вы добавляете код, в котором ожидаете исключение. В вашем приложении это сетевой вызов. В блоке `catch` нужно реализовать код, который предотвращает внезапное завершение приложения. Если возникает исключение, то блок `catch` выполняется для устранения ошибки, а не для резкого завершения работы приложения.

В `getMarsPhotos()`, внутри блока запуска, добавьте блок `try` вокруг вызова `MarsApi` для обработки исключений.

- После блока `try` добавьте блок `catch`.

```
import java.io.IOException  
  
viewModelScope.launch {  
    try {  
        val listResult = MarsApi.retrofitService.getPhotos()  
        marsUiState = listResult  
    } catch (e: IOException) {  
  
    }  
}
```

- Запустите приложение еще раз. Обратите внимание, что на этот раз приложение не аварийно.

Добавление состояния пользовательского интерфейса

В классе `MarsViewModel` статус последнего веб-запроса, `marsUiState`, сохраняется как мутабельный объект состояния. Однако в этом классе отсутствует возможность сохранения различных статусов: загрузки, успеха и неудачи.

- Состояние загрузки указывает на то, что приложение ожидает данных.
- Состояние успеха указывает на то, что данные были успешно получены из веб-сервиса.
- Состояние ошибки указывает на ошибки сети или соединения.

Чтобы представить эти три состояния в приложении, вы используете герметичный интерфейс. Герметичный интерфейс позволяет легко управлять состоянием, ограничивая возможные значения. В приложении `Mars Photos` вы ограничиваете веб-ответ `marsUiState` тремя состояниями (объектами класса данных): загрузка, успех и ошибка, что выглядит следующим образом:

```
// No need to copy over
sealed interface MarsUiState {
    data class Success : MarsUiState
    data class Loading : MarsUiState
    data class Error : MarsUiState
}
```

В приведенном выше фрагменте кода в случае успешного ответа вы получаете от сервера информацию о фотографии Марса. Чтобы сохранить эти данные, добавьте параметр конструктора к классу данных `Success`.

В случае состояний `Loading` и `Error` вам не нужно задавать новые данные и создавать новые объекты; вы просто передаете веб-ответ. Измените класс данных на `Object`, чтобы создать объекты для веб-ответов.

- Откройте файл `ui/MarsViewModel.kt`. После операторов импорта добавьте герметичный интерфейс `MarsUiState`. Это дополнение делает значения, которые может иметь объект `MarsUiState`, исчерпывающими.

```
sealed interface MarsUiState {
    data class Success(val photos: String) : MarsUiState
    object Error : MarsUiState
    object Loading : MarsUiState
}
```

- Внутри класса `MarsViewModel` обновите определение `marsUiState`. Измените тип на `MarsUiState` и `MarsUiState.Loading` в качестве значения по умолчанию. Сделайте сеттер приватным, чтобы защитить запись в `marsUiState`.

```
var marsUiState: MarsUiState by mutableStateOf(MarsUiState.Loading)
private set
```

- Прокрутите страницу вниз до метода `getMarsPhotos()`. Обновите значение `marsUiState` до `MarsUiState.Success` и передайте `listResult`.

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = MarsUiState.Success(listResult)
```

- Внутри блока `catch` обработайте ответ на ошибку. Установите `MarsUiState` в значение `Error`.

```
catch (e: IOException) {
    marsUiState = MarsUiState.Error
}
```

Вы можете вынести присвоение `marsUiState` из блока `try-catch`. Ваша завершенная функция должна выглядеть следующим образом:

```
private fun getMarsPhotos() {
    viewModelScope.launch {
        marsUiState = try {
            val listResult = MarsApi.retrofitService.getPhotos()
            MarsUiState.Success(listResult)
        } catch (e: IOException) {
            MarsUiState.Error
        }
    }
}
```

- В файле `screens/HomeScreen.kt` добавьте выражение `when` для `marsUiState`. Если `marsUiState` равно `MarsUiState.Success`, вызовите `ResultScreen` и передайте `marsUiState.photos`. Пока что игнорируйте ошибки.

```
import androidx.compose.foundation.layout.fillMaxWidth

fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
) {
    when (marsUiState) {
        is MarsUiState.Success -> ResultScreen(
            marsUiState.photos, modifier = modifier.fillMaxWidth()
        )
    }
}
```

Примечание: свойство `marsUiState` больше не является строкой. Вы изменили его на герметичный интерфейс `MarsUiState`, который может иметь три различных значения объекта: `MarsUiState.Loading`, `MarsUiState.Success` и `MarsUiState.Error`.

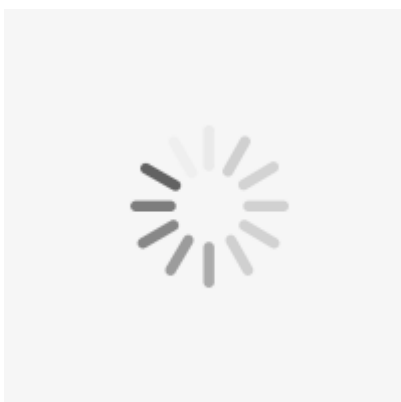
Внутри блока `when` добавьте проверки для `MarsUiState.Loading` и `MarsUiState.Error`. Пусть приложение отобразит композиты `LoadingScreen`, `ResultScreen` и `ErrorScreen`, которые вы реализуете позже.

```
import androidx.compose.foundation.layout.fillMaxSize

fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
) {
    when (marsUiState) {
        is MarsUiState.Loading -> LoadingScreen(modifier = modifier.fillMaxSize())
        is MarsUiState.Success -> ResultScreen(
            marsUiState.photos, modifier = modifier.fillMaxWidth()
        )
        is MarsUiState.Error -> ErrorScreen(modifier = modifier.fillMaxSize())
    }
}
```

Примечание: Если вы реализуете интерфейс `MarsUiState` без ключевого слова `sealed`, это потребует от вас добавить ветви `Success`, `Error`, `Loading` и `else`. Поскольку четвертого варианта (`else`) нет, вы используете интерфейс `sealed`, чтобы сообщить компилятору, что есть только три варианта (тем самым делая условия исчерпывающими).

- Откройте файл `res/drawable/loading_animation.xml`. Этот `drawable` представляет собой анимацию, которая вращает изображение `drawable`, `loading_img.xml`, вокруг центральной точки. (В предварительном просмотре анимация не видна).



- В файле `screens/HomeScreen.kt`, ниже композита `HomeScreen`, добавьте следующую композитную функцию `LoadingScreen` для отображения анимации загрузки. Ресурс `loading_img` `drawable` включен в стартовый код.

```
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.Image

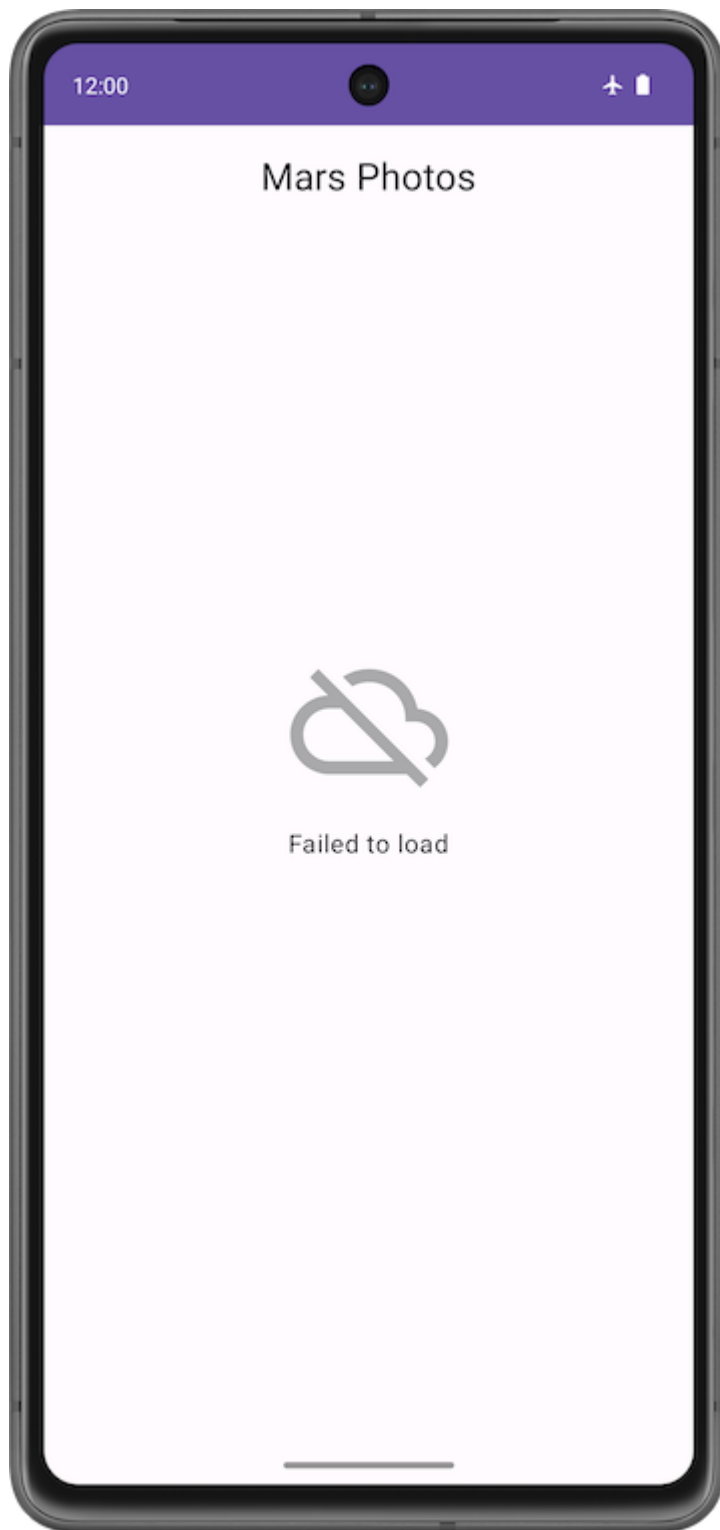
@Composable
fun LoadingScreen(modifier: Modifier = Modifier) {
    Image(
        modifier = modifier.size(200.dp),
        painter = painterResource(R.drawable.loading_img),
        contentDescription = stringResource(R.string.loading)
    )
}
```

- Ниже композитной функции LoadingScreen добавьте следующую композитную функцию **ErrorScreen**, чтобы приложение могло отображать сообщение об ошибке.

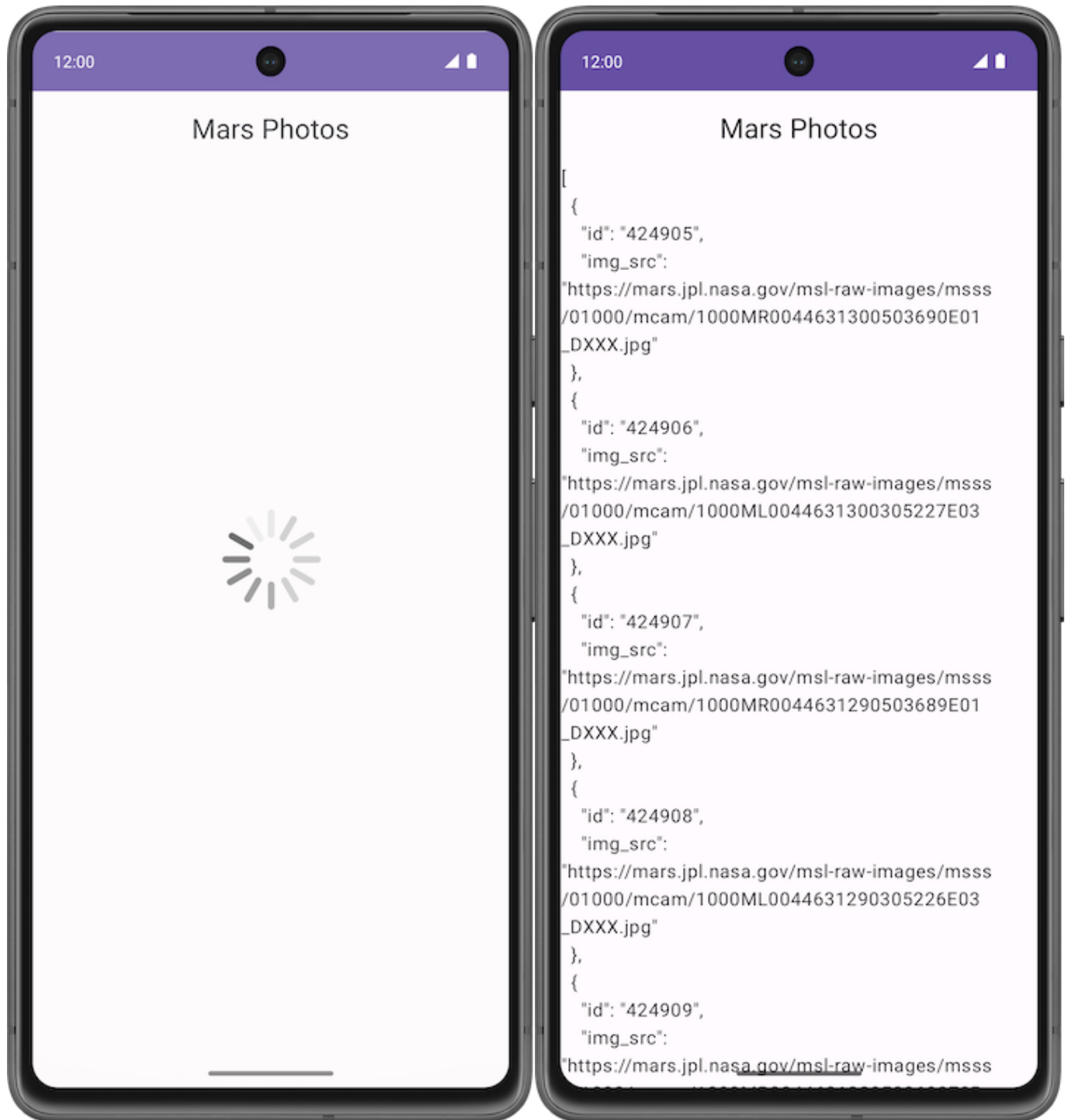
```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding

@Composable
fun ErrorScreen(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier,
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(id = R.drawable.ic_connection_error),
            contentDescription = ""
        )
        Text(text = stringResource(R.string.loading_failed), modifier =
            Modifier.padding(16.dp))
    }
}
```

- Снова запустите приложение с включенным режимом самолета. На этот раз приложение не закрывается внезапно, а выводит следующее сообщение об ошибке:



- Отключите режим самолета на телефоне или эмуляторе. Запустите и протестируйте свое приложение, чтобы убедиться, что все работает правильно и вы можете видеть строку JSON.

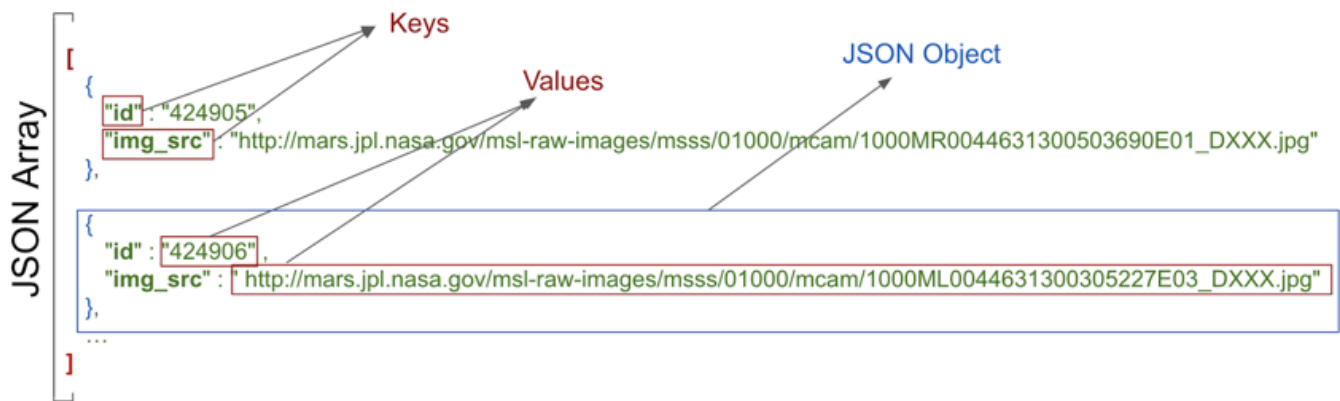


Разбор JSON-ответа с помощью `kotlinx.serialization`

JSON Запрашиваемые данные обычно оформляются в одном из распространенных форматов данных, например XML или JSON. Каждый вызов возвращает структурированные данные, и ваше приложение должно знать, что это за структура, чтобы прочитать данные из ответа.

Например, в этом приложении вы получаете данные с сервера [https:// android-kotlin-fun-mars-server.appspot.com/photos](https://android-kotlin-fun-mars-server.appspot.com/photos). Когда вы вводите этот URL в браузере, вы видите список идентификаторов и URL изображений поверхности Марса в формате JSON!

Структура примера JSON-ответа



Структура ответа JSON имеет следующие особенности:

- JSON-ответ представляет собой массив, обозначенный квадратными скобками. Массив содержит объекты JSON.
- Объекты JSON окружены фигурными скобками.
- Каждый объект JSON содержит набор пар ключ-значение, разделенных запятой.
- Двоеточие разделяет ключ и значение в паре.
- Имена окружены кавычками.
- Значениями могут быть числа, строки, булевы выражения, массивы, объекты (JSON-объекты) или null. Например, `img_src` - это URL, который является строкой. Если вставить этот URL в веб-браузер, вы увидите изображение поверхности Марса.



Теперь в вашем приложении вы получаете JSON-ответ от веб-сервиса Mars, и это отличное начало. Но на самом деле для отображения изображений вам нужны объекты Kotlin, а не большая строка JSON. Этот процесс называется **десериализацией**.

Сериализация - это процесс преобразования данных, используемых приложением, в формат, который можно передавать по сети. В отличие от сериализации, десериализация - это процесс чтения данных из внешнего источника (например, с сервера) и преобразования их в объект времени выполнения. Они оба являются важными компонентами большинства приложений, которые обмениваются данными по сети.

`kotlinx.serialization` предоставляет набор библиотек, которые преобразуют JSON-строку в объекты Kotlin. Существует разработанная сообществом сторонняя библиотека, которая работает с `Retrofit`, `Kotlin Serialization Converter`.

В этом задании вы используете библиотеку `kotlinx.serialization`, чтобы разобрать ответ JSON от веб-сервиса на полезные объекты Kotlin, представляющие фотографии Марса. Вы измените приложение так, чтобы вместо отображения необработанного JSON оно показывало количество полученных фотографий Марса.

- Добавьте зависимости библиотеки `kotlinx.serialization`
- Откройте `build.gradle.kts` (Модуль :app).
- В блоке `plugins` добавьте плагин сериализации `kotlinx`.

```
id('org.jetbrains.kotlin.plugin.serialization') version '1.8.10'
```

В разделе зависимостей добавьте следующий код, чтобы включить зависимость `kotlinx.serialization`. Эта зависимость обеспечивает сериализацию JSON для проектов Kotlin.

```
// Kotlin serialization implementation("org.jetbrains.kotlin:kotlinx-serialization-json:1.5.1")
```

Найдите в блоке зависимостей строку для скалярного конвертера `Retrofit` и измените ее на использование `kotlinx-serialization-converter`:

- Замените следующий код

```
// Retrofit with scalar Converter implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

со следующим кодом

```
// Retrofit with Kotlin serialization Converter implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter:1.0.0") implementation("com.squareup.okhttp3:okhttp:4.11.0")
```

Нажмите **Sync Now**, чтобы перестроить проект с новыми зависимостями.

Примечание: В проекте могут появиться ошибки компилятора, связанные с удаленной скалярной зависимостью `Retrofit`. Вы исправите их на следующих шагах.

Реализуйте класс данных `Mars Photo`

Пример записи ответа в формате JSON, полученного от веб-сервиса, выглядит примерно так, как вы видели ранее:

```
[
  {
    "id": "424906",
    "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631300305227E03_DXXX.jpg"
  },
  ...]
```

В приведенном выше примере обратите внимание, что каждая запись о фотографии Mars содержит следующие пары ключей и значений JSON:

- `id`: идентификатор свойства в виде строки. Поскольку он заключен в кавычки (« »), он имеет тип `String`, а не `Integer`.
- `img_src`: URL-адрес изображения в виде строки.
-

`kotlinx.serialization` разбирает эти JSON-данные и преобразует их в объекты Kotlin. Для этого `kotlinx.serialization` должен иметь класс данных Kotlin для хранения результатов разбора. В этом шаге вы

создадите класс данных `MarsPhoto`.

- Щелкните правой кнопкой мыши на сетевом пакете и выберите `New > Kotlin File/Class`.
- В диалоговом окне выберите `Class` и введите `MarsPhoto` в качестве имени класса. Это действие создаст новый файл `MarsPhoto.kt` в сетевом пакете.
- Сделайте `MarsPhoto` классом данных, добавив ключевое слово `data` перед определением класса.
- Замените фигурные скобки `{}` на круглые скобки `()`. Это изменение приводит к ошибке, поскольку классы данных должны иметь хотя бы одно определенное свойство.

```
data class MarsPhoto()
```

- Добавьте следующие свойства в определение класса `MarsPhoto`.

```
data class MarsPhoto(  
    val id: String, val img_src: String  
)
```

Плагины для сериализации. Добавьте на уровне проекта в `build.gradle.kts`

```
plugins { ... id("org.jetbrains.kotlin.plugin.serialization") version "2.1.0" apply false }
```

на уровне модуля

```
plugins { id("org.jetbrains.kotlin.plugin.serialization") }
```

Зависимости для Retrofit и сериализация

```
// Retrofit implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter:1.0.0")  
implementation("com.squareup.retrofit2:retrofit:2.11.0") implementation("io.coil-kt:coil-compose:2.7.0")  
implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.8.0")
```

Чтобы сделать класс `MarsPhoto` сериализуемым, аннотируйте его с помощью `@Serializable`.

```
import kotlinx.serialization.Serializable  
  
@Serializable  
data class MarsPhoto(  
    val id: String, val img_src: String  
)
```

Обратите внимание, что каждая переменная в классе `MarsPhoto` соответствует имени ключа в объекте JSON. Чтобы соответствовать типам в нашем конкретном JSON-ответе, вы используете объекты `String` для всех значений.

Когда сериализация `kotlinx` разбирает JSON, она сопоставляет ключи по именам и заполняет объекты данных соответствующими значениями.

Аннотация @SerializedName

Иногда имена ключей в JSON-ответе могут создавать путаницу в свойствах Kotlin или не соответствовать рекомендуемому стилю кодирования. Например, в JSON-файле в ключе `img_src` используется символ подчеркивания, в то время как в Kotlin для свойств используются прописные и строчные буквы (верблюжий регистр).

Чтобы использовать в классе данных имена переменных, отличающиеся от имен ключей в JSON-ответе, воспользуйтесь аннотацией `@SerializedName`. В следующем примере имя переменной в классе данных - `imgSrc`. Переменная может быть сопоставлена с атрибутом JSON `img_src` с помощью `@SerializedName(value = «img_src»)`.

- Замените строку для ключа `img_src` строкой, показанной ниже.

```
import kotlinx.serialization.SerializedName

@SerializedName(value = "img_src")
val imgSrc: String
```

Обновление MarsApiService и MarsViewModel

В этом задании вы будете использовать конвертер `kotlinx.serialization` для преобразования JSON-объектов в объекты Kotlin.

- Откройте файл `network/MarsApiService.kt`. Обратите внимание на ошибки неразрешенных ссылок для `ScalarsConverterFactory`. Эти ошибки являются результатом изменения зависимости Retrofit в предыдущем разделе.
- Удалите импорт для `ScalarConverterFactory`. Другую ошибку вы исправите позже.

Удалите:

```
import retrofit2.converter.scalars.ScalarsConverterFactory
```

- В объявлении объекта Retrofit измените конструктор Retrofit, чтобы он использовал `kotlinx.serialization` вместо `ScalarConverterFactory`.

```
import
com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
import kotlinx.serialization.json.Json
import okhttp3.MediaType

private val retrofit = Retrofit.Builder()

    .addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
        .baseUrl(BASE_URL)
        .build()
```

Примечание: Если Android Studio покажет вам какое-либо предупреждение относительно `kotlinx.serialization.ExperimentalSerializationApi`, не обращайтесь на него внимания. Вам не стоит беспокоиться об этом предупреждении. Оно будет устранено в последующих версиях Kotlin.



```
private val retrofit = Retrofit.Builder()
    .addConverterFactory(Json.asConverterFactory(MediaType.get("application/json")))
    .baseUrl(BASE_URL)
    .build()
```

This declaration needs opt-in. Its usage should be marked with '@kotlinx.serialization.ExperimentalSerializationApi' or '@OptIn(kotlinx.serialization.ExperimentalSerializationApi::class)'

Opt in for 'ExperimentalSerializationApi' on 'retrofit' More actions...

Теперь, когда у вас есть `kotlinx.serialization`, вы можете попросить Retrofit вернуть список объектов `MarsPhoto` из массива JSON, вместо того чтобы возвращать строку JSON.

- Обновите интерфейс `MarsApiService` для Retrofit, чтобы он возвращал список объектов `MarsPhoto`, а не строку.

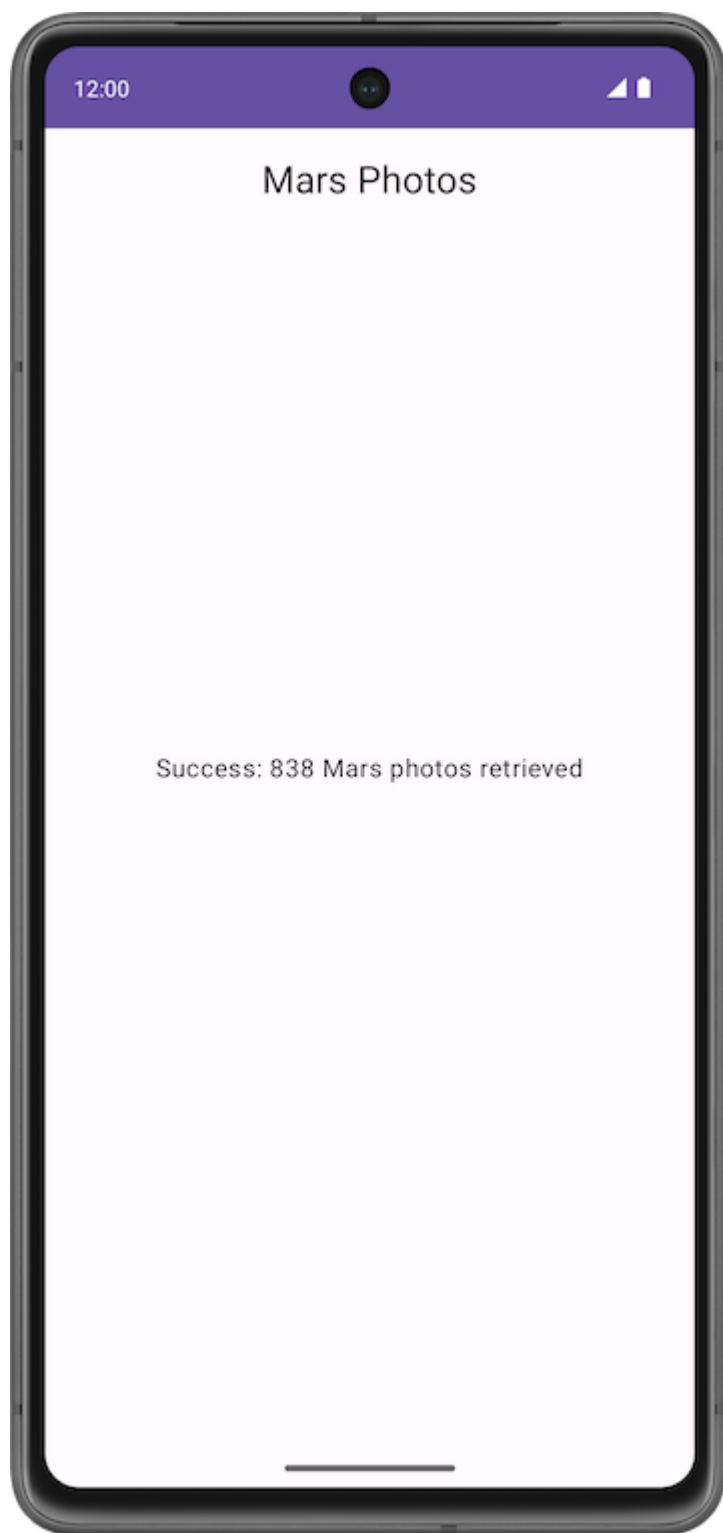
```
interface MarsApiService {
    @GET("photos")
    suspend fun getPhotos(): List<MarsPhoto>
}
```

- Внесите аналогичные изменения в `viewModel`. Откройте файл `MarsViewModel.kt` и прокрутите вниз до метода `getMarsPhotos()`. В методе `getMarsPhotos()` `listResult` - это `List<MarsPhoto>`, а больше не `String`. Размер этого списка - это количество фотографий, которые были получены и разобраны.
- Чтобы вывести количество полученных фотографий, обновите `marsUiState` следующим образом:

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = MarsUiState.Success(
    "Success: ${listResult.size} Mars photos retrieved"
)
```

- Убедитесь, что на устройстве или в эмуляторе выключен режим «В самолете». Скомпилируйте и запустите приложение.
-

На этот раз в сообщении должно быть показано количество свойств, возвращенных веб-сервисом, а не большая строка JSON:



Примечание: Если подключение к Интернету не работает, убедитесь, что на устройстве или эмуляторе выключен режим «В самолете».

Резюме

Веб-сервисы REST

- Веб-сервис - это программная функциональность, предоставляемая через Интернет, которая позволяет вашему приложению делать запросы и получать данные обратно.

- Обычные веб-сервисы используют архитектуру REST. Веб-сервисы, использующие архитектуру REST, называются **RESTful**-сервисами. Веб-сервисы RESTful строятся с использованием стандартных веб-компонентов и протоколов.
- Вы делаете запрос к веб-сервису REST стандартным образом через URI.
- Чтобы использовать веб-сервис, приложение должно установить сетевое соединение и установить связь с сервисом. Затем приложение должно получить и разобрать ответные данные в формате, который приложение может использовать.
- Библиотека **Retrofit** - это клиентская библиотека, которая позволяет вашему приложению делать запросы к веб-сервису REST.
- Используйте конвертеры, чтобы указать Retrofit, что делать с данными, которые он отправляет в веб-сервис и получает от него обратно. Например, конвертер **ScalarsConverter** обрабатывает данные веб-сервиса как строку или другой примитив.
- Чтобы ваше приложение могло подключаться к интернету, добавьте разрешение «android.permission.INTERNET» в манифест Android.
- Ленивая инициализация делегирует создание объекта при первом его использовании. При этом создается ссылка, но не сам объект. Когда к объекту обращаются в первый раз, создается ссылка и используется каждый раз после этого.

Разбор JSON

- Ответ от веб-сервиса часто оформляется в формате JSON - распространенном формате представления структурированных данных.
- Объект JSON - это коллекция пар ключ-значение.
- Коллекция объектов JSON - это массив JSON. Вы получаете массив JSON в качестве ответа от веб-службы.
- Ключи в паре ключ-значение окружены кавычками. Значения могут быть числами или строками. В Kotlin инструменты для сериализации данных доступны в отдельном компоненте `kotlinx.serialization`. `kotlinx.serialization` предоставляет наборы библиотек, которые преобразуют JSON-строки в объекты Kotlin.
- Для Retrofit существует разработанная сообществом библиотека Kotlin Serialization Converter: `retrofit2-kotlinx-serialization-converter`. `kotlinx.serialization` сопоставляет ключи в JSON-ответе со свойствами в объекте данных, которые имеют одинаковые имена.
- Чтобы использовать другое имя свойства для ключа, аннотируйте это свойство аннотацией `@SerializedName` и значением ключа JSON.